

ANALYSIS OF ALGORITHMS

Master Guidance PDF

Dynamic Programming · Memoization · Greedy · Graphs · NP-Completeness · Approximation

C++ implementations · Top-Down & Bottom-Up · Time & Space Complexities · Step-by-Step Theory

CHAPTER 1: DYNAMIC PROGRAMMING

Core Concepts

Dynamic Programming (DP) solves complex problems by breaking them into overlapping subproblems, storing results to avoid recomputation (memoization / tabulation). Two conditions must hold: **Optimal Substructure** and **Overlapping Subproblems**.

Two Approaches:

- **Top-Down (Memoization):** Recursive + cache. Natural to write, may have stack overhead.
- **Bottom-Up (Tabulation):** Iterative table. Better space/speed in practice.

1.1 0/1 Knapsack Problem

Given n items each with weight $w[i]$ and value $v[i]$, and a knapsack of capacity W , find the maximum value achievable without exceeding capacity. Each item is used at most once.

Recurrence: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w[i]] + v[i])$

Top-Down C++ (Memoization):

```
#include <bits/stdc++.h>
using namespace std;
int n, W;
int wt[105], val[105];
int memo[105][1005];
int knapsack(int i, int cap){
    if(i==0 || cap==0) return 0;
    if(memo[i][cap] != -1) return memo[i][cap];
    if(wt[i-1] > cap)
        return memo[i][cap] = knapsack(i-1, cap);
    return memo[i][cap] = max(knapsack(i-1, cap),
        val[i-1] + knapsack(i-1, cap-wt[i-1]));
}
int main(){
    cin >> n >> W;
    for(int i=0; i<n; i++) cin >> wt[i] >> val[i];
    memset(memo, -1, sizeof(memo));
    cout << knapsack(n, W) << endl;
}
```

Bottom-Up C++ (Tabulation):

```
int dp[105][1005];
// dp[i][w] = max value using first i items with capacity w
for(int i=1; i<=n; i++){
    for(int w=0; w<=W; w++){
        dp[i][w] = dp[i-1][w];
        if(wt[i-1] <= w)
            dp[i][w] = max(dp[i][w], val[i-1] + dp[i-1][w-wt[i-1]]);
    }
}
cout << dp[n][W];
```

Approach	Time	Space	Note
Top-Down	$O(n \cdot W)$	$O(n \cdot W) + \text{stack}$	Recursion depth up to n
Bottom-Up	$O(n \cdot W)$	$O(n \cdot W)$	Can optimise to $O(W)$ space

1.2 Coin Change (Min Coins)

Given coin denominations and amount S, find minimum coins needed. Classic unbounded knapsack variant.

Recurrence: $dp[i] = \min \text{ over all coin } c: dp[i-c] + 1$

Bottom-Up:

```
vector<int> dp(S+1, INT_MAX);
dp[0]=0;
for(int i=1;i<=S;i++)
    for(int c : coins)
        if(c<=i && dp[i-c]!=INT_MAX)
            dp[i]=min(dp[i], dp[i-c]+1);
cout << (dp[S]==INT_MAX ? -1 : dp[S]);
```

Top-Down:

```
int memo[10001];
int coinChange(int rem){
    if(rem==0) return 0;
    if(memo[rem]!=-1) return memo[rem];
    int res=INT_MAX;
    for(int c:coins)
        if(c<=rem){
            int sub=coinChange(rem-c);
            if(sub!=INT_MAX) res=min(res,sub+1);
        }
    return memo[rem]=res;
}
```

Approach	Time	Space	Note
Top-Down	$O(S \cdot n)$	$O(S) + \text{stack}$	n = number of coin types
Bottom-Up	$O(S \cdot n)$	$O(S)$	Preferred; no recursion

1.3 Rod Cutting

Rod of length n. Price $p[i]$ for piece of length i. Find max revenue by cutting rod optimally.

Recurrence: $r[n] = \max \text{ over } i=1..n: p[i] + r[n-i]$

Bottom-Up:

```
vector<int> r(n+1,0);
for(int j=1;j<=n;j++)
    for(int i=1;i<=j;i++)
        r[j]=max(r[j], p[i]+r[j-i]);
cout << r[n];
```

Top-Down:

```
int memo[1001];
int cut(int len){
    if(len==0) return 0;
    if(memo[len]!=-1) return memo[len];
    int q=INT_MIN;
    for(int i=1;i<=len;i++)
        q=max(q, p[i]+cut(len-i));
    return memo[len]=q;
}
```

Approach	Time	Space	Note
----------	------	-------	------

Top-Down	$O(n^2)$	$O(n) + \text{stack}$	
Bottom-Up	$O(n^2)$	$O(n)$	

1.4 Maximum Subarray Sum (Kadane's Algorithm)

Find contiguous subarray with largest sum. Kadane's is $O(n)$ DP.

State: $dp[i] = \text{max subarray ending at index } i = \max(a[i], dp[i-1] + a[i])$

```
// Bottom-Up (Kadane)
int maxSub(vector<int>& a){
    int cur=a[0], best=a[0];
    for(int i=1;i<a.size();i++){
        cur=max(a[i], cur+a[i]);
        best=max(best, cur);
    }
    return best;
}
// Top-Down equivalent
int dp[100005], memo_set[100005]={};
int solve(int i, vector<int>&a){
    if(i==0) return a[0];
    if(memo_set[i]) return dp[i];
    memo_set[i]=1;
    return dp[i]=max(a[i], solve(i-1,a)+a[i]);
}
```

Approach	Time	Space	Note
Bottom-Up (Kadane)	$O(n)$	$O(1)$	Optimal
Top-Down	$O(n)$	$O(n)$	Recursion stack

1.5 Longest Common Subsequence (LCS)

Find length of longest subsequence present in both strings X and Y.

Recurrence: if $X[i] == Y[j]$: $dp[i][j] = dp[i-1][j-1] + 1$ else: $\max(dp[i-1][j], dp[i][j-1])$

Bottom-Up:

```
int dp[1001][1001];
for(int i=1;i<=m;i++){
    for(int j=1;j<=n;j++){
        if(X[i-1]==Y[j-1]) dp[i][j]=dp[i-1][j-1]+1;
        else dp[i][j]=max(dp[i-1][j], dp[i][j-1]);
    }
}
cout << dp[m][n];
```

Top-Down:

```
int memo[1001][1001];
int lcs(int i, int j){
    if(i==0||j==0) return 0;
    if(memo[i][j]!=-1) return memo[i][j];
    if(X[i-1]==Y[j-1]) return memo[i][j]=1+lcs(i-1,j-1);
    return memo[i][j]=max(lcs(i-1,j), lcs(i,j-1));
}
```

Approach	Time	Space	Note
Top-Down	$O(m \cdot n)$	$O(m \cdot n) + \text{stack}$	

Bottom-Up	$O(m \cdot n)$	$O(m \cdot n)$	Can reduce to $O(n)$ space
-----------	----------------	----------------	----------------------------

1.6 Chain Matrix Multiplication

Given chain of matrices $A_1 \cdot A_2 \cdot \dots \cdot A_n$, find parenthesization minimizing scalar multiplications.

Recurrence: $m[i][j] = \min \text{ over } k: m[i][k] + m[k+1][j] + p[i-1] \cdot p[k] \cdot p[j]$

Bottom-Up:

```
// p[] has n+1 dimensions; p[i-1]x p[i] is matrix A_i
int m[101][101]={};
for(int len=2; len<=n; len++){
    for(int i=1; i<=n-len+1; i++){
        int j=i+len-1;
        m[i][j]=INT_MAX;
        for(int k=i; k<=j; k++){
            m[i][j]=min(m[i][j], m[i][k]+m[k+1][j]+(long long)p[i-1]*p[k]*p[j]);
        }
    }
}
cout << m[1][n];
```

Top-Down:

```
int memo[101][101];
int mcm(int i, int j){
    if(i==j) return 0;
    if(memo[i][j]!=-1) return memo[i][j];
    memo[i][j]=INT_MAX;
    for(int k=i; k<=j; k++){
        memo[i][j]=min(memo[i][j],
            mcm(i, k)+mcm(k+1, j)+(long long)p[i-1]*p[k]*p[j]);
    }
    return memo[i][j];
}
```

Approach	Time	Space	Note
Top-Down	$O(n^3)$	$O(n^2) + \text{stack}$	
Bottom-Up	$O(n^3)$	$O(n^2)$	

1.7 Optimal Binary Search Tree

Keys $k_1 < k_2 < \dots < k_n$ with search probabilities $p[i]$ and dummy-key probabilities $q[i]$. Build BST minimising expected search cost.

$e[i][j] = \min \text{ over } r=i..j: e[i][r-1] + e[r+1][j] + w[i][j]$ where w = sum of probabilities

Bottom-Up (CLRS §15.5):

```
double e[105][105], w[105][105];
// e[i][i-1]=q[i-1], w[i][i-1]=q[i-1]
for(int l=1; l<=n; l++){
    for(int i=1; i<=n-l+1; i++){
        int j=i+l-1;
        e[i][j]=DBL_MAX;
        w[i][j]=w[i][j-1]+p[j]+q[j];
        for(int r=i; r<=j; r++){
            double t=e[i][r-1]+e[r+1][j]+w[i][j];
            if(t<e[i][j]) e[i][j]=t;
        }
    }
}
```

```
cout << e[1][n];
```

Approach	Time	Space	Note
Bottom-Up	$O(n^3)$	$O(n^2)$	Standard CLRS approach
Top-Down	$O(n^3)$	$O(n^2)$	Memoize $e[i][j]$

CHAPTER 2: MEMOIZATION — Overview via Fibonacci

Memoization = top-down DP. Store result of each subproblem in a cache (array/map). On re-call, return cached value instead of recomputing.

Fibonacci Example:

```
// Naive Recursive - O(2^n)
int fib(int n){ return n<=1?n:fib(n-1)+fib(n-2); }
// Memoized - O(n) time, O(n) space
int memo[101]={-1};
int fibMemo(int n){
    if(n<=1) return n;
    if(memo[n]!=-1) return memo[n];
    return memo[n]=fibMemo(n-1)+fibMemo(n-2);
}
// Bottom-Up - O(n) time, O(1) space
int fibDP(int n){
    if(n<=1) return n;
    int a=0,b=1;
    for(int i=2;i<=n;i++){int c=a+b;a=b;b=c;}
    return b;
}
```

Key Insight: Memoization turns exponential recursion into polynomial by caching.

Use memoization when: recursion is natural, not all subproblems needed.

Use tabulation when: all subproblems needed, better constant factors, limited stack.

Approach	Time	Space	Note
Naive Recursive	$O(2^n)$	$O(n)$ stack	Recomputes same values
Top-Down Memo	$O(n)$	$O(n)$ + stack	Cache array
Bottom-Up	$O(n)$	$O(1)$	Best for Fibonacci

CHAPTER 3: GREEDY ALGORITHMS

Greedy algorithms make locally optimal choices at each step, hoping to reach global optimum. No backtracking. Faster than DP but not always correct — proof of correctness must exist.

Greedy vs DP:

- **DP:** Considers all possibilities for each subproblem. Always correct if optimal substructure holds.
- **Greedy:** Makes one irreversible choice. Correct only when greedy-choice property holds.

3.1 Activity Selection Problem

Select maximum non-overlapping activities from set. Each has start $s[i]$ and finish $f[i]$. Greedy: always pick activity with earliest finish time.

```
// Sort by finish time, then greedily select
vector<pair<int,int>> acts; // {finish, start}
sort(acts.begin(),acts.end());
int last=-1, count=0;
for(auto [f,s]:acts){
    if(s>=last){ count++; last=f; }
}
cout << count;
```

Approach	Time	Space	Note
Greedy	$O(n \log n)$	$O(1)$	Sorting dominates; provably optimal

3.2 Fractional Knapsack (Greedy)

Items can be split. Greedy: sort by value/weight ratio descending, take as much as possible.

Note: 0/1 Knapsack requires DP; Fractional Knapsack is solved greedily.

```
struct Item{ double v,w; };
bool cmp(Item a, Item b){ return a.v/a.w > b.v/b.w; }
double fracKnap(vector<Item>& items, double W){
    sort(items.begin(),items.end(),cmp);
    double total=0;
    for(auto& it:items){
        if(W<=0) break;
        double take=min(it.w,W);
        total+=take*(it.v/it.w);
        W-=take;
    }
    return total;
}
```

Approach	Time	Space	Note
Greedy	$O(n \log n)$	$O(1)$	Provably optimal via exchange argument

3.3 Travelling Salesman Problem — Greedy Approximation

Visit all cities exactly once, return to start, minimise total distance. Greedy (Nearest Neighbour): always go to closest unvisited city. Not optimal — can be arbitrarily bad — but fast $O(n^2)$.

```
int dist[105][105];
bool visited[105]={};
```

```

int tsp_greedy(int n, int start){
    int cur=start, total=0;
    visited[start]=true;
    for(int step=1;step<n;step++){
        int best=-1, bd=INT_MAX;
        for(int j=0;j<n;j++){
            if(!visited[j] && dist[cur][j]<bd)
                bd=dist[cur][j], best=j;
            visited[best]=true;
            total+=bd; cur=best;
        }
        total+=dist[cur][start]; // return
    }
    return total;
}

```

Approach	Time	Space	Note
Greedy Nearest Neighbour	$O(n^2)$	$O(n)$	No optimality guarantee

3.4 Huffman Encoding

Optimal prefix-free variable-length encoding. Greedy: repeatedly merge two lowest-frequency nodes.

Steps: (1) Build min-heap of characters by frequency. (2) Extract two minimum nodes, create parent with sum frequency. (3) Insert parent back. Repeat until one node remains (root).

```

struct Node{
    char ch; int freq;
    Node *left,*right;
};
struct Cmp{
    bool operator()(Node* a,Node* b){return a->freq>b->freq;}
};
Node* buildHuffman(map<char,int>& freq){
    priority_queue<Node*,vector<Node*>,Cmp> pq;
    for(auto [c,f]:freq)
        pq.push(new Node{c,f,nullptr,nullptr});
    while(pq.size()>1){
        Node* l=pq.top(); pq.pop();
        Node* r=pq.top(); pq.pop();
        pq.push(new Node{'#',l->freq+r->freq,l,r});
    }
    return pq.top();
}

```

Approach	Time	Space	Note
Huffman Build	$O(n \log n)$	$O(n)$	Min-heap operations
Encoding/Decode	$O(L)$	$O(n)$	L = total encoded length

No Correctness Check in Greedy:

Greedy algorithms lack automatic correctness verification. You must PROVE correctness via (a) Greedy-Choice Property and (b) Optimal Substructure. Example: Dijkstra fails on negative edges because greedy choice becomes invalid.

CHAPTER 4: ELEMENTARY GRAPH ALGORITHMS

4.1 Representations of Graphs

Two main representations for graph $G = (V, E)$:

Adjacency Matrix: 2D array $A[u][v] = 1$ if edge exists. $O(V^2)$ space. $O(1)$ edge lookup. Good for dense graphs.

Adjacency List: Array of lists. $O(V+E)$ space. $O(\text{degree})$ edge lookup. Good for sparse graphs.

```
// Adjacency Matrix
int adjMatrix[1005][1005] = {};
void addEdge(int u, int v){ adjMatrix[u][v]=adjMatrix[v][u]=1; }
// Adjacency List
vector<int> adjList[1005];
void addEdgeL(int u, int v){
    adjList[u].push_back(v);
    adjList[v].push_back(u); // undirected
}
```

Property	Adj Matrix	Adj List
Space	$O(V^2)$	$O(V+E)$
Edge check	$O(1)$	$O(\text{deg}(u))$
Add edge	$O(1)$	$O(1)$
Iterate neighbours	$O(V)$	$O(\text{deg}(u))$
Best for	Dense graphs	Sparse graphs

4.2 Breadth-First Search (BFS)

Explores graph level by level using a queue. Finds shortest path (in terms of edges) in unweighted graphs. Assigns colours: WHITE (unvisited), GRAY (discovered), BLACK (processed).

Applications: Shortest path, Level-order traversal, Bipartite check, Connected components.

```
void bfs(int src, vector<int> adj[], int V){
    vector<int> dist(V+1, -1), parent(V+1, -1);
    queue<int> q;
    dist[src]=0; q.push(src);
    while(!q.empty()){
        int u=q.front(); q.pop();
        for(int v:adj[u]){
            if(dist[v]==-1){
                dist[v]=dist[u]+1;
                parent[v]=u;
                q.push(v);
            }
        }
    }
    // dist[v] = shortest path length from src to v
}
```

Approach	Time	Space	Note
BFS	$O(V+E)$	$O(V)$	Queue + visited array

4.3 Depth-First Search (DFS)

Explores as deep as possible before backtracking. Records discovery/finish timestamps. Classifies edges: Tree, Back, Forward, Cross.

Applications: Topological sort, Cycle detection, SCC (Kosaraju/Tarjan), Articulation points.

```
int timer_=0;
int disc[1005],fin[1005],color[1005];
// 0=WHITE,1=GRAY,2=BLACK
void dfs_visit(int u, vector<int> adj[]){
    color[u]=1; disc[u]=++timer_;
    for(int v:adj[u]){
        if(color[v]==0) dfs_visit(v,adj);
        // color[v]==1 => Back edge (cycle!)
    }
    color[u]=2; fin[u]=++timer_;
}
void dfs(int V, vector<int> adj[]){
    for(int u=1;u<=V;u++)
        if(color[u]==0) dfs_visit(u,adj);
}
// Topological Sort: output vertices in reverse finish order
```

Approach	Time	Space	Note
DFS	$O(V+E)$	$O(V)$	Stack (implicit recursion)

CHAPTER 5: NP-COMPLETENESS — Overview

This chapter provides a theoretical overview. No C++ code required — focus on concepts and reductions.

5.1 Complexity Classes

Class	Definition	Example
P	Solvable in polynomial time	Sorting, BFS, Shortest Path
NP	Verifiable in polynomial time; may not be solvable in poly time	Hamiltonian Cycle, SAT
NP-Complete	In NP + every NP problem reduces to it in poly time	3-SAT, TSP, Knapsack (decision)
NP-Hard	At least as hard as NP-Complete; may not be in NP	Halting problem

5.2 Key Contrasting Problem Pairs

Shortest Path vs Longest Simple Path: Shortest path (Dijkstra/BFS) is P. Longest simple path is NP-Hard — no polynomial algorithm known.

Euler Tour vs Hamiltonian Cycle: Euler Tour (visit every EDGE once) is P — solvable in $O(E)$. Hamiltonian Cycle (visit every VERTEX once) is NP-Complete.

CNF-SAT vs 3-CNF-SAT: Both NP-Complete. Cook-Levin theorem showed CNF-SAT is NP-Complete first. 3-SAT (clauses with exactly 3 literals) is the workhorse for reductions.

5.3 Polynomial Reductions

To show problem B is NP-Complete: (1) Show B is in NP (verifiable in poly time). (2) Take known NP-Complete problem A and show $A \leq_p B$ (A reduces to B in poly time). If A is hard and A reduces to B, then B is at least as hard as A.

Reduction Chain: Circuit-SAT $\rightarrow$ CNF-SAT $\rightarrow$ 3-SAT $\rightarrow$ Clique $\rightarrow$ Vertex Cover $\rightarrow$ Hamiltonian Cycle $\rightarrow$ TSP

5.4 Hamiltonian Cycle vs TSP

Hamiltonian Cycle: Decision problem — does a cycle visiting every vertex exactly once exist? NP-Complete.

TSP (Travelling Salesman): Optimisation — find MINIMUM cost Hamiltonian cycle. NP-Hard. Decision version (Is there tour $\leq k$?) is NP-Complete.

Reduction: Ham-Cycle $\leq_p$ TSP-decision by setting edge weights 0 for existing edges, 1 for missing.

P = NP? The million-dollar open question. If $P=NP$, all NP-Complete problems become tractable.

CHAPTER 6: APPROXIMATION ALGORITHMS —

Overview

Since NP-Complete problems lack polynomial exact solutions (assuming $P \neq NP$), approximation algorithms find near-optimal solutions with guaranteed quality bounds.

Approximation Ratio: Algorithm A has ratio $\rho(n)$ if for all inputs, $\max(C/C^*, C^*/C) \leq \rho(n)$ where C is algorithm's cost and C^* is optimal cost.

Key Approximation Algorithms:

Problem	Algorithm	Ratio	Complexity
Vertex Cover	Greedy Matching	2	$O(V+E)$
TSP (metric)	Christofides	1.5	$O(n^3)$
TSP (metric)	MST heuristic	2	$O(E \log V)$
Set Cover	Greedy	$\ln n + 1$	$O(n^3)$
Max 3-SAT	Random Assignment	8/7	$O(n)$

Vertex Cover Approximation (2-approx):

Repeatedly pick any uncovered edge (u,v) , add both u and v to cover, remove all incident edges. Result $\leq 2 \times \text{OPT}$ because each edge in the matching must be covered by at least one vertex in OPT.

TSP 2-Approximation (Metric TSP):

(1) Build MST using Prim/Kruskal — $O(E \log V)$. (2) DFS preorder walk gives Hamiltonian path. (3) By triangle inequality, shortcutting repeated vertices only decreases cost. Result $\leq 2 \times \text{OPT}$.

PTAS and FPTAS:

PTAS (Polynomial Time Approximation Scheme): For any $\epsilon > 0$, achieves $(1+\epsilon)$ -approximation in poly time.
Example: Euclidean TSP.

FPTAS (Fully PTAS): Also polynomial in $1/\epsilon$. Example: Knapsack (scaled DP).

QUICK REFERENCE: ALL ALGORITHMS

Algorithm	Type	Time	Space	Key Idea
0/1 Knapsack	DP	$O(nW)$	$O(nW)$	Include/exclude each item
Coin Change	DP	$O(Sn)$	$O(S)$	Min coins for each amount
Rod Cutting	DP	$O(n^2)$	$O(n)$	Best cut at each length
Max Subarray	DP	$O(n)$	$O(1)$	Kadane's extend/restart
LCS	DP	$O(mn)$	$O(mn)$	Match chars or skip
Chain Matrix	DP	$O(n^3)$	$O(n^2)$	Try all split points
Optimal BST	DP	$O(n^3)$	$O(n^2)$	Try each key as root
Fibonacci	Memo	$O(n)$	$O(n)$	Cache recursive calls
Activity Select	Greedy	$O(n \log n)$	$O(1)$	Earliest finish first
Frac Knapsack	Greedy	$O(n \log n)$	$O(1)$	Best v/w ratio first
Huffman	Greedy	$O(n \log n)$	$O(n)$	Merge min-freq pair
BFS	Graph	$O(V+E)$	$O(V)$	Level-by-level queue
DFS	Graph	$O(V+E)$	$O(V)$	Deep-first recursion
Vertex Cover	Approx	$O(V+E)$	$O(V)$	2-approx via matching
TSP MST	Approx	$O(E \log V)$	$O(V)$	2-approx via MST tour

SPACE COMPLEXITY — Step-by-Step Guidelines

Step 1 — Count the data structures:

List every array, vector, map, matrix. Note their sizes in terms of input parameters (n , W , m , etc.).

Step 2 — Count the recursion stack:

For recursive algorithms, depth of recursion adds $O(\text{depth})$ stack space. DFS on tree: $O(h)$. Unbounded recursion: $O(n)$.

Step 3 — Identify if space can be reused:

Many DP tables only need previous row $\rightarrow$ reduce from $O(n^2)$ to $O(n)$. Example: LCS, Knapsack can use $O(W)$ or $O(n)$ space.

Step 4 — Ignore constants:

Space complexity is asymptotic. $O(2n) = O(n)$. $O(n + 50) = O(n)$.

Step 5 — Auxiliary vs Total:

Auxiliary space = extra space excluding input. Total space = input + auxiliary. Algorithms using in-place sorting have $O(1)$ auxiliary space.

Common Space Complexities:

- $O(1)$: Kadane's, Fractional Knapsack, Greedy selection — only scalar variables used.
- $O(n)$: 1D DP arrays, BFS/DFS visited array, recursion stack of linear depth.

EXAM STRATEGY & KEY TAKEAWAYS

DP Problems — 4-Step Method:

- (1) Define subproblem clearly with indices.
- (2) Write recurrence relation.
- (3) Identify base cases.
- (4) Determine evaluation order (for bottom-up) or memoize.

Greedy — Proving Correctness:

- Greedy-Choice Property: Locally optimal choice leads to globally optimal.
- Optimal Substructure: After greedy choice, remaining subproblem is still optimal.
- Exchange Argument: Show swapping greedy choice for another doesn't improve.

NP-Completeness — What to Remember:

- $P \subseteq NP$. NP-Complete = hardest problems in NP. No polynomial algorithm known if $P \neq NP$.
- To prove NP-Complete: membership in NP + polynomial reduction from known NPC problem.
- Key problems: 3-SAT, Vertex Cover, Clique, Hamiltonian Cycle, TSP-decision.

Graph Algorithms — When to Use:

- BFS → shortest path (unweighted), level traversal, bipartite check.
- DFS → topological sort, cycle detection, SCC, connectivity.

Approximation — Key Ratios:

- Vertex Cover: 2-approx. Metric TSP: 2-approx (MST) or 1.5 (Christofides). Set Cover: $O(\log n)$.

Good luck on your exam! Master the recurrences and you master DP.